Homework no. 3

Let $\boldsymbol{n}$ be the size of the linear system, $\boldsymbol{\varepsilon}$ - the computations error, the square matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and a vector $\boldsymbol{s} \in \mathbb{R}^{n}$.

1. Compute vector $\boldsymbol{b} \in \mathbb{R}^{n}$ in the following way:

$$b_i = \sum_{j=1}^{n} s_j a_{ij} \quad , \; i = 1, \ldots, n$$

2. Implement the $\boldsymbol{QR}$ decomposition for matrix $\boldsymbol{A}$ using Householder's algorithm.
3. Solve the linear system:

$$Ax = b \quad ,$$

   using the $\boldsymbol{QR}$ decomposition implemented in one of the libraries mention on the lab webpage (one obtains the solution $\boldsymbol{x_{QR}}$) and the $\boldsymbol{QR}$ decomposition computed at item 2. (one obtains the solution $\boldsymbol{x_{Householder}}$). Compute and display:

$$\left\| x_{QR} - x_{Householder} \right\|_2 .$$

4. Compute and display the following errors ($\| \cdot \|_2$ is the Euclidean norm):

$$\| A^{init} x_{Householder} - b^{init} \|_2 \quad ,$$

$$\| A^{init} x_{QR} - b^{init} \|_2 \quad ,$$

$$\frac{\| x_{Householder} - s \|_2}{\| s \|_2} \quad ,$$

$$\frac{\| x_{QR} - s \|_2}{\| s \|_2} .$$

   (these values should be smaller than $\mathbf{10^{-6}}$)
5. Compute the inverse matrix of matrix $\boldsymbol{A}$ using the $\boldsymbol{QR}$ decomposition computed at item 2. Compare it with inverse matrix computed using the corresponding function from the library. Display the norm:

$$\left\| A^{-1}_{Householder} - A^{-1}_{bibl} \right\| .$$

6. Write your program with the possibility of random initialization of the input data (thus your program can run for any value of $\boldsymbol{n}$).

Solving linear systems using a *QR* decomposition

Let $\boldsymbol{A}$ be a real, square matrix of size $\boldsymbol{n}$. Assume that for matrix $\boldsymbol{A}$ one has a decomposition of the following form:

$$A=Q*R$$

where $\boldsymbol{Q}$ is an orthogonal matrix ($Q^TQ = QQ^T = I_n$) and $\boldsymbol{R}$ is a upper triangular matrix. If we have such a decomposition for matrix $\boldsymbol{A}$, solving the linear system $\boldsymbol{Ax=b}$ is equivalent with solving the upper triangular system $\boldsymbol{Rx=Q^Tb}$.

$$Ax=b \;\leftrightarrow\; Q*Rx = b \;\leftrightarrow\; Q^T*Q*Rx = Q^T b \;\leftrightarrow\; Rx=Q^T b$$

$$Ax=b \quad \Leftrightarrow \quad Rx=Q^T b$$

Householder's Algorithm

For transforming the linear system $\boldsymbol{Ax=b}$ into $\boldsymbol{Rx = Q^Tb}$ one uses the reflection matrices. A reflection matrix $\boldsymbol{P = (p_{ij})_{i,j=1,n}}$ has the following form:

$$P = I_n - 2vv^T \quad , \quad v \in \mathbb{R}^n, \; \|v\|_2 = |v| = \sqrt{\sum_{i=1}^{n} v_i^2} = 1.$$

(we denoted by $\boldsymbol{I_n}$ the unity matrix of size $\boldsymbol{n}$)

One can show that the reflection matrices are symmetric and orthogonal:

$$P = P^T \quad , \quad P^2 = I_n .$$

Householder's algorithm for computing the $\boldsymbol{QR}$ decomposition has $\boldsymbol{(n-1)}$ steps. In step $\boldsymbol{r}$ the column $\boldsymbol{r}$ of matrix $\boldsymbol{A}$ is transformed in upper triangular form without modifying the first $\boldsymbol{(r-1)}$ columns. In this step one also obtains column $\boldsymbol{r}$ of matrix $\boldsymbol{R}$. One can compute and store matrix $\boldsymbol{R}$ directly in matrix $\boldsymbol{A}$ (one can say that matrix $\boldsymbol{A}$ is transformed in an upper triangular matrix). In the same time, one can perform the necessary transformations on vector $\boldsymbol{b}$ to obtain $\boldsymbol{Q^Tb}$ one can also compute matrix $\boldsymbol{Q^T}$. For computing the matrix $\boldsymbol{Q^T}$, one start with $\bar{Q} = I_n$ and then we perform the same transformations on matrix $\bar{Q}$ as those performed on matrix $\boldsymbol{A}$.

Step ***r (r=1,2,…,n-1)***

When entering this step, matrix $\boldsymbol{A}$ has the first $\boldsymbol{(r\text{-}1)}$ columns in upper triangular form. In this step we want to transform column $\boldsymbol{r}$ of matrix $\boldsymbol{A}$ in upper triangular form (without modifying the upper triangular form of the first $\boldsymbol{(r\text{-}1)}$ columns). To do this, one uses a reflection matrix $\boldsymbol{P_r}$:

$$\begin{aligned} \boldsymbol{A} &= \boldsymbol{P_r} * \boldsymbol{A} \\ \boldsymbol{b} &= \boldsymbol{P_r} * \boldsymbol{b} \\ \bar{\boldsymbol{Q}} &= \boldsymbol{P_r} * \bar{\boldsymbol{Q}} \end{aligned}$$

where matrix $\boldsymbol{P_r}$ is computed in the following way:

$$\boldsymbol{P_r} = \boldsymbol{I_n} - \frac{\mathbf{1}}{\boldsymbol{\beta}} \boldsymbol{u}\boldsymbol{u}^T$$

$$\boldsymbol{\beta} = \boldsymbol{\sigma} - \boldsymbol{k}\boldsymbol{a_{rr}}, \;\; \boldsymbol{\sigma} = \sum_{j=r}^{n} \boldsymbol{a_{jr}^2} \;, \;\; \boldsymbol{k} = -\mathrm{sign}(\boldsymbol{a_{rr}})\sqrt{\boldsymbol{\sigma}}\,, \;\; \mathbf{sign}(\boldsymbol{x}) = \begin{cases} +\mathbf{1} & \mathbf{if}\ \boldsymbol{x} \geq \mathbf{0}, \\ -\mathbf{1} & \mathbf{if}\ \boldsymbol{x} < \mathbf{0}. \end{cases}$$

$$\boldsymbol{u} = \begin{pmatrix} \mathbf{0} \\ \vdots \\ \mathbf{0} \\ \boldsymbol{a_{rr}} - \boldsymbol{k} \\ \boldsymbol{a_{r+1r}} \\ \vdots \\ \boldsymbol{a_{nr}} \end{pmatrix}, \qquad \begin{aligned} \boldsymbol{u_i} &= \mathbf{0},\ \boldsymbol{i} = \mathbf{1},\ldots,\boldsymbol{r} - \mathbf{1}, \\ \boldsymbol{u_r} &= \boldsymbol{a_{rr}} - \boldsymbol{k}, \\ \boldsymbol{u_i} &= \boldsymbol{a_{ir}}, \boldsymbol{i} = \boldsymbol{r} + \mathbf{1},\ldots,\boldsymbol{n} \end{aligned}$$

Matrix $\boldsymbol{V} = \boldsymbol{u}\boldsymbol{u}^T \in \mathbb{R}^{n \times n}$, $\boldsymbol{V} = \left(\boldsymbol{v_{ij}}\right)_{\boldsymbol{i},\boldsymbol{j}=\mathbf{1},\ldots,\boldsymbol{n}}$ has the following elements:

$$\boldsymbol{v_{ij}} = \begin{cases} \mathbf{0} & \text{for } \boldsymbol{i} = \mathbf{1},\ldots,\boldsymbol{r} - \mathbf{1}\,,\ \boldsymbol{j} = \mathbf{1},\ldots,\boldsymbol{n} \\ \mathbf{0} & \text{for } \boldsymbol{i} = \boldsymbol{r},\ldots,\boldsymbol{n}\,,\ \boldsymbol{j} = \mathbf{1},\ldots,\boldsymbol{r} - \mathbf{1} \\ \boldsymbol{u_i}\boldsymbol{u_j} & \text{for } \boldsymbol{i} = \boldsymbol{r},\ldots,\boldsymbol{n}\,,\ \boldsymbol{j} = \boldsymbol{r},\ldots,\boldsymbol{n} \end{cases}$$

The reflection matrix $\boldsymbol{P_r} = (\boldsymbol{p_{ij}})_{i,j=1,n}$ has the elements:

$$p_{ij} = \begin{cases} 0 & \text{for } i = 1,\ldots,r-1\,,\ \ j = 1,\ldots,n,\ i \neq j \\ 1 & \text{for } i = 1,\ldots,r-1,\ j = i \\ 0 & \text{for } i = r,\ldots,n\,,\ \ j = 1,\ldots,r-1 \\ -f\,u_i u_j & \text{for } i = r,\ldots,n\,,\ \ j = r,\ldots,n,\ i \neq j \\ 1 - f\,u_i^2 & \text{for } i = r,\ldots,n\,,\ j = i \end{cases} \quad , \quad f = \frac{1.0}{\beta}$$

After applying this algorithm, the upper triangular matrix $\boldsymbol{R}$ is stored in matrix $\boldsymbol{A}$, and the transpose of matrix $\boldsymbol{Q}$ can be found in matrix $\bar{\boldsymbol{Q}}$. In vector $\boldsymbol{b}$ we'll have $\boldsymbol{Q^T b^{init}}$. The verification that the initial matrix is non-singular ($\det \boldsymbol{A} \neq 0$) is done by testing if all the diagonal elements of matrix $\boldsymbol{R}$ (or $\boldsymbol{A}$) are non-zero ($|r_{ii}| \leq \varepsilon,\ \forall i$)
.

Householder's algorithm for computing the ***A=QR*** *factorization*

$Q = I_n;$

for $r = 1,\ldots,n-1$

// **building of matrix** P_r **– constant** β **and vector** u

- $\sigma = \sum_{i=r}^{n} a_{ir}^2;$
- **if** ($\sigma \le \varepsilon$) **break** ; // $r = r+1 \leftrightarrow P_r = I_n$ (A **singular**)
- $k = \sqrt{\sigma};$
- **if** ($a_{rr} > 0$) $k = -k;$
- $\beta = \sigma - k\, a_{rr};$
- $u_r = a_{rr} - k;\ \ u_i = a_{ir}\ , i = r+1,\ldots,n;$

// $A = P_r * A$

// **columns** $j = r+1,\ldots,n$ **transformation**

- **for** $j = r+1,\ldots,n$

  $* \gamma = (\gamma_j / \beta) = (Ae_j, u) / \beta = (\sum_{i=r}^{n} u_i a_{ij}) / \beta;$

  $*$ **for** $i = r,\ldots,n$

  $a_{ij} = a_{ij} - \gamma * u_i;$

// **column** r **of matrix** A **transformation**

- $a_{rr} = k;\ a_{ir} = 0,\ \ i = r+1,\ldots,n;$

// $b = P_r * b$

- $\gamma = (\gamma / \beta) = (b, u) / \beta = (\sum_{i=r}^{n} u_i b_i) / \beta;$
- **for** $i = r,\ldots,n \quad b_i = b_i - \gamma * u_i;$

// $\widetilde{Q = P_r} * \widetilde{Q}$

- **for** $j = 1,\ldots,n$

  $* \gamma = (\widetilde{Qe_j}, u) / \beta = (\sum_{i=r}^{n} u_i \widetilde{q_{ij}}) / \beta;$

  $*$ **for** $i = r,\ldots,n$

  $\widetilde{q_{ij}} = \widetilde{q_{ij}} - \gamma * u_i;$

Computing an approximation for the inverse of a matrix

Assume we know a numerical method for solving linear systems, in our situation the QR decomposition provided by the Householder algorithm. Then the columns of the inverse matrix can be approximated by solving $\boldsymbol{n}$ linear systems.

Column $\boldsymbol{j}$ of matrix $\boldsymbol{A^{-1}}$ is approximated by solving the linear system:

$$\boldsymbol{Ax = e_j} \ , \ \boldsymbol{e_j = (0,0,\ldots, \underset{\text{position } j}{1}, \ldots, 0)^T} \ , \ \boldsymbol{j = 1,\ldots,n} \ .$$

The procedure of computing matrix $\boldsymbol{A^{-1}_{Householder}}$ is the following:

- Compute the $\boldsymbol{QR}$ decomposition of matrix $\boldsymbol{A}$ using Householder's algorithm.
- If matrix $\boldsymbol{A}$ is singular (det $\boldsymbol{A} = 0 \leftrightarrow \exists\ |\boldsymbol{r_{ii}}| < \boldsymbol{\varepsilon}$), STOP, the inverse matrix cannot be computed.
- Else, compute the columns of matrix $\boldsymbol{A^{-1}_{Householder}}$ in the following way:

  for $\boldsymbol{j=1,..,n}$
  1. Set vector $\boldsymbol{b = Q^T e_j}$ = column $\boldsymbol{j}$ of matrix $\boldsymbol{Q^T}$ or row $\boldsymbol{j}$ of matrix $\boldsymbol{Q}$;
     (use the initialization with the column $\boldsymbol{j}$ of matrix $\boldsymbol{Q^T}$ or row $\boldsymbol{j}$ of matrix $\boldsymbol{Q}$ depending on the matrix returned by the Householder algorithm)
  2. Solve the upper triangular system $\boldsymbol{Rx=b}$, using the back substitution. One obtains the solution $\boldsymbol{x}^*$ ($\boldsymbol{x}^*$ is also the solution of the linear system $\boldsymbol{Ax=e_j}$);
  3. Store $\boldsymbol{x}^*$ in column $\boldsymbol{j}$ of matrix $\boldsymbol{A^{-1}_{Householder}}$.

The procedure described above is a numerical method that uses the $\boldsymbol{QR}$ decomposition, for solving the matrix equation:

$$\boldsymbol{AX=I_n} \ .$$

Example

$$A=\begin{pmatrix}0&0&4\\1&2&3\\0&1&2\end{pmatrix},\quad s=\begin{pmatrix}3\\2\\1\end{pmatrix}$$

$$\begin{array}{rcrcrcr} & & & & 4x_3 & = & 4\\ x_1 & + & 2x_2 & + & 3x_3 & = & 10\\ & & x_2 & + & 2x_3 & = & 4\end{array}$$

$$A=\begin{pmatrix}0&0&4\\1&2&3\\0&1&2\end{pmatrix}=QR=\begin{pmatrix}0&0&1\\1&0&0\\0&1&0\end{pmatrix}*\begin{pmatrix}1&2&3\\0&1&2\\0&0&4\end{pmatrix},\ b=A*s=\begin{pmatrix}4\\10\\4\end{pmatrix}$$

$$Ax=b\Leftrightarrow Rx=Q^Tb\Leftrightarrow \begin{array}{rcrcrcr} x_1 & + & 2x_2 & + & 3x_3 & = & 10\\ & & x_2 & + & 2x_3 & = & 4\\ & & & & 4x_3 & = & 4\end{array}\Rightarrow x^*=\begin{pmatrix}3\\2\\1\end{pmatrix}$$

The first column of matrix $A^{-1}_{Householder}$

$$\begin{array}{rcrcrcr} x_1 & + & 2x_2 & + & 3x_3 & = & 0\\ & & x_2 & + & 2x_3 & = & 0\\ & & & & 4x_3 & = & 1\end{array}\Rightarrow x^*=\begin{pmatrix}0.25\\-0.5\\0.25\end{pmatrix}$$

Second column of matrix $A^{-1}_{Householder}$

$$\begin{array}{rcrcrcr} x_1 & + & 2x_2 & + & 3x_3 & = & 1\\ & & x_2 & + & 2x_3 & = & 0\\ & & & & 4x_3 & = & 0\end{array}\Rightarrow x^*=\begin{pmatrix}1.0\\0.0\\0.0\end{pmatrix}$$

Third column of matrix $A^{-1}_{Householder}$

$$\begin{array}{rcrcrcl} x_1 & + & 2x_2 & + & 3x_3 & = & 0 \\ & & x_2 & + & 2x_3 & = & 1 \\ & & & & 4x_3 & = & 0 \end{array} \Rightarrow x^* = \begin{pmatrix} -2.0 \\ 1.0 \\ 0.0 \end{pmatrix}$$

$$A^{-1}_{Householder} = \begin{pmatrix} 0.25 & 1.0 & -2.0 \\ -0.5 & 0.0 & 1.0 \\ 0.25 & 0.0 & 0.0 \end{pmatrix}$$